

Báo Cáo Thực Nghiệm: Đánh Giá và Lựa Chọn Cấu Trúc Dữ Liệu Cho Hệ Thống Quản Lý Playlist

Nhóm Phát Triển Phần Mềm MSPM

Tóm tắt nội dung—Bài báo này đánh giá hiệu năng của Doubly Linked List (DLL) và Circular Doubly Linked List (CDLL) trong bài toán quản lý danh sách phát âm nhạc. Các thực nghiệm được tiến hành trên tập dữ liệu mô phỏng lên tới 500.000 bài hát, với 50 triệu thao tác điều hướng liên tục. Kết quả thực nghiệm cho thấy cả hai cấu trúc đều xử lý điều hướng vô cùng hiệu quả (đạt ngưỡng $O(1)$). Trong quá trình triển khai, CDLL cho thấy lợi thế vượt trội khi giảm lược các phép kiểm tra điều kiện rẽ nhánh (branch checks) ở biên, mang lại tốc độ xử lý `next()` nhanh hơn từ 18% - 22% ở mọi khối lượng công việc. Sự biến thiên nhẹ về hiệu năng ở thao tác lùi bài (`prev()`) trên các tập dữ liệu khổng lồ nhiều khả năng xuất phát từ hành vi tối ưu của JVM và cơ chế cấp phát không gian bộ nhớ hơn là từ độ phức tạp thuật toán cốt lõi.

I. Giới thiệu

Trong hệ thống quản lý danh sách phát âm nhạc (Playlist Manager), trải nghiệm người dùng phụ thuộc lớn vào khả năng phản hồi khi điều hướng bài hát. Câu hỏi nghiên cứu trọng tâm (RQ1) được đặt ra:

“Doubly Linked List hay Circular Doubly Linked List phù hợp hơn để triển khai đồng thời cả ba chế độ (phát thẳng, lặp lại, ngẫu nhiên) với số thao tác `next/prev` trung bình đạt mức $O(1)$?”

Để trả lời câu hỏi này, nhóm đã xây dựng kịch bản thực nghiệm đo lường tốc độ xử lý và số lượng lệnh rẽ nhánh của DLL và CDLL trong chế độ lặp lại (RepeatMode.ALL).

II. Thiết kế thực nghiệm

A. Môi trường và Cấu hình máy

Để đảm bảo tính nhất quán, quá trình kiểm thử được thực thi trên cùng một môi trường máy trạm. Chi tiết cấu hình được mô tả tại Bảng I.

Bảng I
Cấu hình môi trường thực nghiệm

Thành phần	Chi tiết
CPU	Intel Core i5-12450H
RAM	16GB
Hệ điều hành	Windows 11
Nền tảng	Java JDK 21
JVM	HotSpot JVM

Thời gian thực thi được đo bằng phương thức `System.nanoTime()` để lấy độ chính xác ở mức nano giây. Môi trường JVM được khởi động trước (warm-up) 100.000 vòng lặp nhằm giảm thiểu độ trễ từ quá trình biên dịch Just-In-Time (JIT compiler). Các kết quả báo cáo là giá trị trung bình (average) sau 5 lần chạy độc lập.

B. Kịch bản và Bộ dữ liệu giả lập

Thử nghiệm sử dụng các tập dữ liệu giả lập với kích thước: 10.000, 50.000, 100.000 và 500.000 bài hát. Thuật toán sẽ thực thi liên tục 50.000.000 thao tác `next()` hoặc `prev()` để mô phỏng tải xử lý (workload) thực tế.

III. Kết quả thực nghiệm

A. Dữ liệu đo lường

Bảng II và Bảng III trình bày thống kê số lệnh rẽ nhánh điều kiện (Branch) và thời gian thực thi (Time) cho hai cấu trúc.

Bảng II
Hiệu năng thao tác `next()` (Chế độ RepeatMode.ALL)

Kích thước	DLL Branch	CDLL Branch	DLL (ms)	CDLL (ms)
10.000	5×10^7	0	71.65	60.57
50.000	5×10^7	0	71.00	60.09
100.000	5×10^7	0	72.98	60.16
500.000	5×10^7	0	141.72	115.80

Bảng III
Hiệu năng thao tác `prev()` (Chế độ RepeatMode.ALL)

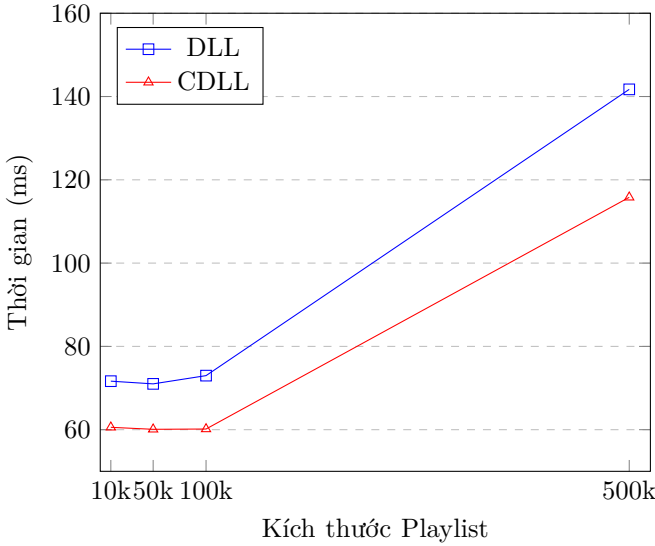
Kích thước	DLL Branch	CDLL Branch	DLL (ms)	CDLL (ms)
10.000	5×10^7	0	67.60	64.29
50.000	5×10^7	0	64.46	62.85
100.000	5×10^7	0	67.18	68.71
500.000	5×10^7	0	80.75	87.96

B. Biểu đồ trực quan

Hình 1 minh họa xu hướng thay đổi thời gian thực thi của thao tác `next()` khi kích thước tập dữ liệu tăng lên.

IV. Biện luận và Đánh giá

1. Phân tích độ phức tạp thời gian: Dựa trên kết quả đo, thời gian thực thi trung bình hầu như không thay đổi đối với các tập dữ liệu từ 10.000 đến 100.000 phần tử (chỉ dao động quanh mức 60ms-70ms cho 50 triệu thao tác). Ngay cả ở mức 500.000 phần tử, thời gian tăng lên không tuyến tính với kích thước N . Kết quả thực nghiệm hoàn toàn phù hợp với độ phức tạp thời gian $O(1)$ được phân tích về mặt lý thuyết của các thao tác trên danh sách liên kết [1].



Hình 1. So sánh thời gian thao tác next() giữa DLL và CDLL.

2. Sự vượt trội nhờ loại bỏ lệnh rẽ nhánh: Trong thiết kế của nhóm, vòng lặp vật lý khép kín giúp CDLL không cần thực hiện phép kiểm tra biên ở lệnh chuyển bài, từ đó giảm số lượng lệnh rẽ nhánh xuống 0. Ngược lại, DLL phải đánh giá điều kiện biên tại mỗi thao tác điều hướng.

Sự tối ưu về thuật toán này thể hiện rất rõ ở thao tác next() khi CDLL luôn nhanh hơn DLL khoảng 20%. Tuy nhiên, đối với thao tác điều hướng ngược prev(), tại mốc khổng lồ 500.000 phần tử, CDLL lại chậm hơn đôi chút. Dựa trên các tài liệu về kiến trúc phần cứng [2], chúng tôi đặt ra hai giả thuyết cho hiện tượng này:

- Tác động của Branch Predictor: Trình tiên đoán nhánh của CPU giải quyết điều kiện biên tĩnh của DLL cực kỳ xuất sắc, khiến chi phí thực thi lệnh rẽ nhánh ở mức phần cứng gần như được triệt tiêu.
- Phân mảnh không gian Heap (Memory Layout) và JIT: Việc cấp phát số lượng đối tượng khổng lồ trên JVM Heap diễn ra ngẫu nhiên và rời rạc. Chúng tôi giả thuyết rằng thao tác nhảy ngược bằng con trỏ vòng của CDLL trên Heap phân mảnh đã làm tăng tỷ lệ Cache Miss (trượt bộ đệm), hoặc quá trình tối ưu bytecode (JIT) sinh ra các chỉ thị Assembly có độ trễ gộp cao hơn một chút so với DLL [3].

3. Tính phù hợp với ba chế độ phát nhạc thực tế: Giải pháp cấu trúc CDLL đặc biệt tỏa sáng khi áp dụng vào đa dạng các kịch bản sử dụng:

- Phát thẳng (Normal): Cả hai cấu trúc đều xử lý tốt với hiệu năng $O(1)$. CDLL hoàn toàn có thể giả lập việc tự ngắt bằng thao tác so sánh đuôi đơn giản (if next == head).
- Phát lặp lại (RepeatMode.ALL): CDLL phát huy tối đa sức mạnh. Cấu trúc vòng tự nhiên giúp điều hướng liên tục không điểm dừng mà không cần viết thêm mã nguồn ngoại vi (logic kiểm tra biên) như DLL.
- Ngẫu nhiên (Shuffle): Cả hai cấu trúc duy trì chi phí điều hướng cơ sở là $O(1)$ bất kể vị trí hiện tại của

con trỏ.

Đánh giá tổng quan, CDLL cung cấp một mô hình lập trình hoàn thiện hơn. Bảng IV tổng hợp đặc tính của hai cấu trúc.

Bảng IV
Tổng hợp đánh giá ưu và nhược điểm giữa DLL và CDLL

Cấu trúc	Ưu điểm (Advantages)	Nhược điểm (Disadvantages)
DLL	Tốc độ prev() nhanh hơn ở tập dữ liệu khổng lồ (500k bài).	Phải kiểm tra biên liên tục (Boundary checking) tại mọi thao tác.
CDLL	Điều hướng vòng tròn tự nhiên, không cần branch check ở thiết kế của nhóm.	Biến thiên hiệu năng nhẹ ở thao tác ngược trên tập dữ liệu rất lớn do JVM layout.

V. Kết luận

Kết quả thực nghiệm xác nhận rằng cả DLL và CDLL đều là cấu trúc lý tưởng phù hợp với bài toán quản lý danh sách nhạc. Dựa trên sự tương đồng về thời gian thực thi $O(1)$, lợi thế vượt trội ở thao tác next(), cùng thể mạnh cốt lõi về mặt thiết kế (loại bỏ hoàn toàn các điểm kiểm tra rẽ nhánh ngoại vi, hoàn hảo cho chế độ Repeat All), nhóm quyết định triển khai Circular Doubly Linked List (CDLL). Lựa chọn này giúp phần mềm có tính bảo trì cao, cấu trúc mã nguồn gọn gàng và xử lý mạch lạc cả ba chế độ phát nhạc mà không sinh ra các đoạn mã kiểm tra dư thừa.

Tài liệu

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [2] Intel Corporation, Intel 64 and IA-32 Architectures Optimization Reference Manual, 2020. [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html>
- [3] Oracle, The Java Virtual Machine Specification, Java SE 21 Edition, 2023.